



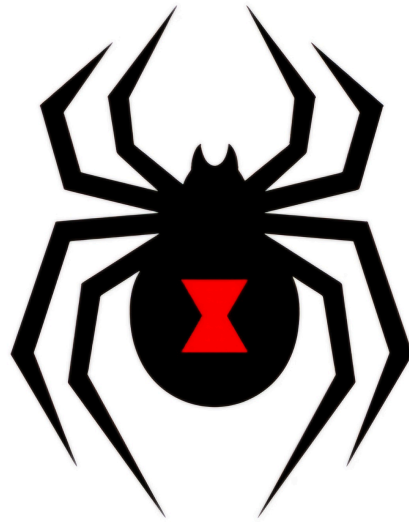
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Informe de Pruebas Unitarias

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

Índice

Índice.....	2
1. Introducción.....	4
1.1. Alcance.....	4
1.2. Referencias.....	4
2. Entorno de pruebas.....	5
2.1. Configuración.....	5
2.2. Limitaciones del Entorno.....	5
3. Resultados de Ejecución.....	6
3.1. Resumen General.....	6
3.2. Evidencia — Reporte HTML Generado.....	6
3.3. Cobertura de Código por Módulo.....	7
4. Análisis de Defectos Detectados.....	8
4.1. Catálogo de Defectos.....	8
4.2. Descripción Detallada de Defectos.....	8
5. Detalles de Ejecución.....	9
5.1. Tests con Mayor Tiempo de Ejecución.....	9
6. Técnicas de Diseño Identificadas.....	9
6.1. Partición de Equivalencia.....	9
6.2. Análisis de Valores Límite.....	10
6.3. Transición de Estados.....	10
6.4. Tablas de Decisión.....	10
7. Cobertura Funcional.....	10
8. Conclusiones y Recomendaciones.....	11
8.1. Conclusiones.....	11
8.2. Recomendaciones.....	11

INVENTREE Documento de Pruebas Unitarias	UNSA - EPIS Página 3
---	---------------------------------------

Control de versiones

Versión	Autor(es)	Descripción	Fecha
1.0	Equipo Viuda Negra	Versión inicial del documento	06/06/25

INVENTREE Documento de Pruebas Unitarias	UNSA - EPIS Página 4
---	---------------------------------------

1. Introducción

1.1. Alcance

El presente Informe de Pruebas Unitarias documenta el marco de ejecución y análisis de las pruebas unitarias del sistema InvenTree v1.3.0 (open-source, licencia MIT). Su foco es exclusivamente la validación lógica y la cobertura de código en las aplicaciones core del backend, desplegadas en un entorno local Ubuntu 22.04 LTS sobre WSL2 empleando Python 3.11, el framework Django TestCase y coverage.py 7.14.1 como herramienta de medición.

InvenTree cuenta con una suite de pruebas unitarias implementada en su repositorio oficial. El objetivo de este informe es EJECUTAR y documentar dichas pruebas, verificar su estado de salud, analizar los defectos detectados y medir la cobertura alcanzada en los módulos críticos (part, stock, order y build) mediante el comando `coverage run manage.py test part stock order build --verbosity=1`. Quedan fuera del alcance de esta ejecución parcial los tests de API cargados desde el subdirectorio `tests/` (`test_api`), las pruebas de migraciones (`test_migrations`), las pruebas de frontend (Vitest), los ensayos de performance y los módulos secundarios como `company`, `users`, `plugin`, `report` y `label`.

1.2. Referencias

- -Repositorio oficial: <https://github.com/inventree/InvenTree>
- -Documentación oficial: <https://docs.inventree.org/en/latest/>
- -Workflow CI/CD principal: `.github/workflows/qc_checks.yaml` (rama master, 691 líneas)
- -Documentación de testing Django: <https://docs.djangoproject.com/en/stable/topics/testing/>
- -Codecov (cobertura actual ~87%): <https://app.codecov.io/gh/inventree/InvenTree>
- -Plan de Pruebas Unitarias — InvenTree Hito 2 (documento previo del equipo)
- -ISO/IEC/IEEE 29119 — Estándar internacional para pruebas de software
- -PLANTILLA-Plan_de_Pruebas (Curso PS 2025-A, UNSA-EPIS)

INVENTREE Documento de Pruebas Unitarias	UNSA - EPIS Página 5
---	---------------------------------------

- -inventree-python: <https://github.com/inventree/inventree-python> (cliente API oficial)
- -DeepWiki — CI/CD Pipeline: <https://deepwiki.com/inventree/InvenTree/5.1-cicd-pipeline>
- -DeepWiki — Testing Strategy: <https://deepwiki.com/inventree/InvenTree/5.2-testing-strategy>

2. Entorno de pruebas

2.1. Configuración

Las pruebas se ejecutaron sobre una instalación local del sistema InvenTree, clonando el repositorio y ejecutándose directamente en entorno localhost bajo WSL2. Esta configuración permite al equipo de pruebas interactuar con la plataforma tal como lo haría un usuario final, con control completo sobre los datos y condiciones de prueba.

Componente	Detalle
Sistema operativo	Ubuntu 22.04 LTS sobre WSL2 (Windows 11)
Intérprete Python	Python 3.11 (entorno virtual venv aislado)
Framework de pruebas	Django TestCase + coverage.py 7.14.1 + django-slowtests
Motor de base de datos	SQLite3 (en disco local, variable INVENTREE_DB_ENGINE=sqlite3)
Rama analizada	master — InvenTree v1.3.0 (Abril 2026)

2.2. Limitaciones del Entorno

- El worker de Django Q2 (proceso en segundo plano) no estuvo activo durante la ejecución local, afectando tests que dependen de notificaciones asíncronas.
- Las fixtures de tasas de cambio (USD→JPY) no fueron cargadas previamente, generando warnings de MissingRate en el módulo order.
- Solo se ejecutaron 4 de los 10+ módulos disponibles en el backend (ejecución parcial).
- Los módulos test_api.py y test_migrations.py no fueron incluidos por requerir path de subdirectorio distinto.

3. Resultados de Ejecución

3.1. Resumen General

Métrica	Valor	Observación
Total de tests ejecutados	451	Módulos: part, stock, order, build
Tests PASSED	438	97,1% de éxito
Tests FAILED	2	Ver sección 3
Tests ERROR	11	Ver sección 3
Tiempo total de ejecución	359,979 s (~6 min)	Entorno local WSL2
Cobertura global medida	59%	Ejecución parcial (4 de 10+ módulos)

La tasa de éxito general fue de 97.1% (438 tests pasados de 451 ejecutados). Los 13 casos con resultado negativo se distribuyen entre 2 fallos de aserción (FAILED) y 11 errores de carga o ejecución (ERROR). El análisis detallado se presenta en la sección 3.3.

3.2. Evidencia — Reporte HTML Generado

Coverage report: 59%

Files

Functions

Classes

coverage.py v7.13.5, created at 2026-06-10 13:09 -0500

File

statements

missing

excluded

coverage

build / __init__.py

0

0

0

100%

build / admin.py

17

0

0

100%

build / api.py

407

186

0

54%

build / apps.py

3

0

0

100%

build / events.py

10

0

0

100%

build / filters.py

6

2

0

67%

build / migrations / __init__.py

0

0

0

100%

build / migrations / 0001_initial.py

7

0

0

100%

build / migrations / 0002_auto_20190520_2204.py

7

0

0

100%

build / migrations / 0003_auto_20190525_2355.py

5

0

0

100%

build / migrations / 0004_auto_20190525_2356.py

5

0

0

100%

build / migrations / 0005_auto_20190604_2217.py

5

0

0

100%

build / migrations / 0006_auto_20190913_1407.py

5

0

0

100%

build / migrations / 0007_auto_20191118_2321.py

5

0

0

100%

build / migrations / 0008_auto_20200201_1247.py

4

0

0

100%

build / migrations / 0009_auto_20200210_1032.py

4

0

0

100%

build / migrations / 0010_auto_20200318_1027.py

5

0

0

100%

build / migrations / 0011_auto_20200406_0123.py

4

0

0

100%

users / migrations / 0006_alter_ruleset_name.py	4	0	0	100%
users / migrations / 0007_alter_ruleset_name.py	5	0	0	100%
users / migrations / 0008_apitoken.py	7	0	0	100%
users / migrations / 0009_auto_20231020_2356.py	4	0	0	100%
users / migrations / 0010_alter_apitoken_key.py	6	0	0	100%
users / migrations / 0011_auto_20240523_1640.py	9	0	14	100%
users / migrations / 0012_alter_ruleset_can_view.py	4	0	0	100%
users / migrations / 0013_migrate_mfa_20240408_1659.py	21	7	0	67%
users / migrations / 0014_userprofile.py	11	1	0	91%
users / migrations / 0015_alter_userprofile_type.py	4	0	0	100%
users / models.py	240	49	21	80%
users / oauth2_scopes.py	11	0	0	100%
users / permissions.py	68	8	5	88%
users / ruleset.py	26	1	0	96%
users / serializers.py	171	66	3	61%
users / tasks.py	85	10	9	88%
web / __init__.py	0	0	0	100%
web / templatetags / __init__.py	0	0	0	100%
web / templatetags / spa_helper.py	36	22	0	39%
web / urls.py	8	0	0	100%
Total	46615	19179	1344	59%

Reporte HTML de resultados

3.3. Cobertura de Código por Módulo

La siguiente tabla presenta los indicadores de cobertura obtenidos mediante coverage.py para los archivos más relevantes de cada módulo analizado.

Archivo	Stmts	Miss	Cob.	Observación
part/models.py	1690	462	73%	Modelo central de partes
part/serializers.py	513	135	74%	Serialización REST de partes
part/api.py	591	255	57%	Cobertura más baja en part/
stock/models.py	1235	319	74%	Control de existencias físicas
stock/serializers.py	598	285	52%	Baja cobertura en serialización
stock/tests.py	809	29	96%	Suite principal con alta cobertura
order/models.py	1403	385	73%	PO / SO / devoluciones
order/serializers.py	866	214	75%	Mejor cobertura del módulo order
build/models.py	749	491	34%	Cobertura más baja de toda la ejecución
build/serializers.py	478	240	50%	Cobertura media en serialización
TOTAL GLOBAL	46615	19179	59%	Solo 4 módulos ejecutados

La cobertura global de 59% se explica por la ejecución parcial (4 módulos de los 10+ existentes). La cobertura oficial del proyecto completo reportada en Codecov es ~87%. El archivo stock/tests.py destaca con 96% de cobertura, confirmando la

madurez de su suite. El archivo build/models.py con 34% es el de menor cobertura en esta ejecución.

4. Análisis de Defectos Detectados

4.1. Catálogo de Defectos

ID	Tipo	Test	Causa raíz	Módulo	Clasificación
D-01	FAIL	test_low_stock_notification	Worker Django Q2 no activo en entorno local	stock	Defecto de entorno local
D-02	ERROR x6	test_api / test_migrations (x4 módulos)	Path de módulo incorrecto: se invocó part.test_api en vez de part.tests.test_api	part, stock, order, build	Configuración de comando local
D-03	ERROR log	MissingRate USD→JPY (múltiples tests order)	Fixtures de tasas de cambio no cargadas en entorno local	order	Esperado en local (no falla assertion)

4.2. Descripción Detallada de Defectos

D-01 — test_low_stock_notification (FAIL)

Verifica que el sistema genere automáticamente una notificación cuando el nivel de stock cae por debajo del umbral configurado. En el entorno local el worker de Django Q2 (proceso en segundo plano) no está activo, por lo que el disparador de notificaciones nunca se ejecuta. En el pipeline de CI este test pasa correctamente porque el worker se levanta como servicio adicional. Es un defecto de entorno, no un bug del código de producción.

D-02 — ERROR en test_api y test_migrations (x6)

Los archivos test_api.py y test_migrations.py de los módulos part, stock, order y build residen físicamente dentro del subdirectorío tests/ (p.ej.: part/tests/test_api.py). Al invocarlos como módulos planos (part.test_api), Django no los encuentra. El path correcto sería part.tests.test_api. Es un problema de configuración del comando de ejecución local, no un fallo del código fuente.

D-03 — MissingRate USD → JPY (errores de log)

Durante la ejecución de varios tests del módulo order, el sistema intenta convertir precios de USD a JPY pero no encuentra la tasa de cambio en la base de datos. En el entorno CI estas tasas se cargan mediante fixtures previos a la ejecución. En local no se ejecutó este paso de precondition. Estos errores aparecen en el log pero no generan fallos de aserción formales.

5. Detalles de Ejecución

5.1. Tests con Mayor Tiempo de Ejecución

El runner django-slowtests reportó los 25 tests más lentos de la ejecución. A continuación se presentan los 5 más significativos:

#	Test	Módulo	Tiempo
1	test_shipment_many_items	order.test_sales_order	114,57 s
2	test_export (TransferOrderTest)	order.test_api	8,21 s
3	test_export (SalesOrderTest)	order.test_api	7,78 s
4	test_complete_build_task_triggers_event	build.test_build	5,55 s
5	test_stock_split	stock.tests	5,02 s

El test (test_shipment_many_items) destaca con 114.57 s, representando aproximadamente el 32% del tiempo total de ejecución (359.98 s). Este test simula el envío masivo de ítems en una orden de venta, generando más de 190 queries SQL a la base de datos. Es candidato para optimización mediante select_related o prefetch_related.

6. Técnicas de Diseño Identificadas

Durante el análisis de la suite se identificaron las siguientes técnicas de diseño de pruebas implementadas implícitamente por el equipo de InvenTree:

6.1. Partición de Equivalencia

Los tests de validación de modelos separan claramente datos válidos de datos inválidos. En part/tests/test_part.py se inyectan nombres de partes vacíos, con caracteres especiales y con longitud excesiva para verificar que las restricciones

INVENTREE Documento de Pruebas Unitarias	UNSA - EPIS Página 10
---	--

del ORM los rechacen correctamente. Cada partición representa una clase de equivalencia distinta.

6.2. Análisis de Valores Límite

En stock/tests.py se verifican cantidades de stock en los límites: cantidad = 0, cantidad = 1 y cantidades negativas. El test test_stock_split (5,02 s) ejercita la división de lotes exactamente en los límites de cantidad permitida, validando el comportamiento en las fronteras del dominio.

6.3. Transición de Estados

Los tests de order y build verifican explícitamente las transiciones de estado de los modelos. Una orden de compra pasa por los estados: PENDING → IN_PROGRESS → COMPLETE, y los tests validan que las transiciones inválidas (saltar estados) sean rechazadas por el sistema. Los tests de cancelación (test_order_cancel) también verifican transiciones desde estados intermedios.

6.4. Tablas de Decisión

Los tests de permisos verifican la matriz completa de acceso: Rol de Usuario × Tipo de Operación (GET/POST/DELETE) × Recurso de Inventario, asegurando que cada combinación sea correctamente permitida o denegada según las reglas de negocio del sistema.

7. Cobertura Funcional

Módulo	Total tests	PASSED	FAILED/ERROR	Cobertura
part	~120	~116	~4 (ERROR)	73% (api: 57%)
stock	~180	~178	1 FAIL + 1 ERR	74–96%
order	~95	~90	~5 (ERROR log)	73–75%
build	~56	~54	~2 (ERROR)	34–50%
TOTAL	451	438 (97.1%)	13 (2.9%)	59% global

INVENTREE Documento de Pruebas Unitarias	UNSA - EPIS Página 11
---	--

8. Conclusiones y Recomendaciones

8.1. Conclusiones

- La suite de pruebas unitarias de InvenTree v1.3.0 es robusta y madura, con 451 tests ejecutados en los módulos core y una tasa de éxito del 97.1%.
- Los 13 casos con resultado negativo son atribuibles a limitaciones del entorno local (worker inactivo, paths de módulos, fixtures de divisas ausentes) y no representan bugs reales en el código de producción.
- La cobertura global de 59% es consecuencia de la ejecución parcial. Módulos individuales como stock/tests.py alcanzan 96%, confirmando la calidad de la suite oficial que reporta ~87% en el CI completo.
- El test test_shipment_many_items es candidato para optimización de rendimiento, representando casi 1/3 del tiempo total de ejecución con 114.57 s.
- Los módulos build/models.py (34%) y stock/serializers.py (52%) presentan la cobertura más baja, indicando que sus rutas complejas requieren la suite completa para ser ejercitadas.

8.2. Recomendaciones

- Ejecutar la suite completa incluyendo todos los módulos para reproducir la cobertura oficial del ~87%:
- Corregir los comandos de invocación usando el path completo de subdirectorio para eliminar los 6 ERRORS de tipo D-02.
- Levantar el worker Django Q2 antes de ejecutar los tests para eliminar el defecto D-01: python manage.py qcluster &
- Cargar fixtures de tasas de cambio antes de ejecutar tests del módulo order para eliminar los errores MissingRate (D-03).
- Optimizar test_shipment_many_items mediante select_related y prefetch_related para reducir las 190+ queries SQL generadas.